

Guia Completo do Projeto

API de Gerenciamento de Documentação de Colaboradores

Repositório Oficial: <https://github.com/rhenanteix/Inmeta-api>

Stack principal: NestJS, TypeScript, PostgreSQL, Prisma ORM, Jest, Pino Logger

1. Objetivo do Sistema

Este projeto consiste no desenvolvimento de uma API REST robusta voltada para a gestão do ciclo de vida documental de colaboradores de uma organização. O sistema resolve o problema de conformidade trabalhista e organizacional ao gerenciar com precisão a obrigatoriedade de documentos, a detecção de pendências de envio, e o versionamento histórico de arquivos, impedindo falhas em transações críticas.

2. Arquitetura de Tecnologias

- **NestJS:** Framework opinativo para a construção de aplicações eficientes e escaláveis no ecossistema Node.js.
- **TypeScript:** Tipagem estática garantindo previsibilidade e redução de bugs em tempo de execução.
- **PostgreSQL:** Banco de dados relacional escolhido pela confiabilidade e suporte total a transações ACID complexas.
- **Prisma ORM:** Abstração do banco com segurança de tipos automatizada de ponta a ponta.
- **Jest:** Suíte de testes utilizada para garantir a cobertura completa de serviços e regras de negócios.
- **Pino Logger:** Logger estruturado de alta performance para auditoria e rastreabilidade em produção.

3. Estrutura de Pastas Padronizada

O design arquitetural segue a convenção modular do NestJS, separando as preocupações comuns das entidades do domínio técnico:

```
src/
├── common/                # Middlewares, interceptors, decorators e dtos
└── compartilhados
    ├── database/
    │   └── prisma/        # Configuração e inicialização do PrismaService
    ├── health/           # Endpoints de monitoramento de saúde do app (Health Check)
    ├── dashboard/        # Agregações de dados e métricas gerenciais
    └── modules/
        ├── employee/     # Módulo de cadastro e ciclo de vida do colaborador
        └── document-type/ # Módulo de definição de regras e tipos de documentos
```

```
└─ employee-document/ # Módulo de tabelas de junção e transações de vínculo
└─ document/         # Módulo de arquivos e controle estrito de versões
```

4. Modelagem de Dados Relacional (Prisma Schema)

Todas as entidades do sistema implementam identificação única por meio de `UUID` gerados nativamente, controle temporal via `createdAt` e `updatedAt`, e exclusão lógica segura por meio do campo `deletedAt` (Soft Delete).

Relacionamentos Lógicos:

- Um Colaborador (`Employee`) possui uma ou mais parametrizações de obrigatoriedade (`EmployeeDocument`) [1:N].
- Um Tipo de Documento (`DocumentType`) está associado a uma ou mais parametrizações (`EmployeeDocument`) [1:N].
- Cada registro de parametrização (`EmployeeDocument`) possui opcionalmente um Documento real enviado (`Document`) [1:1].
- Um Documento (`Document`) armazena múltiplas atualizações históricas organizadas em versões (`DocumentVersion`) [1:N].

Importante sobre o Soft Delete:

Consultas realizadas via Prisma devem filtrar explicitamente por `deletedAt: null` para que registros deletados logicamente não contaminem as estatísticas e as listagens comuns do sistema.

5. Ordem Sugerida de Desenvolvimento

1. Inicialização estrutural da aplicação com o CLI do NestJS.
2. Provisionamento da infraestrutura isolada com Docker Compose (PostgreSQL).
3. Inicialização e configuração do Prisma ORM (`prisma init`).
4. Construção do arquivo `schema.prisma` e execução da primeira migration.
5. Criação do `PrismaService` global no módulo de banco de dados.
6. Implementação do ecossistema CRUD da entidade `Employee`.
7. Implementação do ecossistema CRUD da entidade `DocumentType`.
8. Criação das regras de negócio de vinculação em `EmployeeDocument`.
9. Codificação do motor de upload lógico e versionamento físico de arquivos.
10. Construção do motor de busca exaustiva de pendências sob paginação.
11. Construção das agregações quantitativas do painel do Dashboard.
12. Configuração de Terminus para rotas de Health Check da API.

13. Acomodação do Pino Logger para interceptação global de requisições.
14. Desenvolvimento dos cenários de teste unitários simulados (Mocks).
15. Confeção final do documento explicativo README.md.

6. Regras de Negócio e Transações Atomizadas

A integridade referencial e o comportamento corporativo baseiam-se em restrições explícitas:

- **Bloqueio de Duplicidade:** Um colaborador não pode possuir mais de um vínculo ativo para o exato mesmo tipo de documento. A tentativa dispara um erro HTTP 409 (Conflict).
- **Criação Implícita:** No exato momento em que o primeiro envio de arquivo acontece, o registro pai `Document` é construído por debaixo dos panos, acompanhado de sua respectiva `DocumentVersion` inicial marcada estritamente com o status `ACTIVE`.
- **Estrito Controle de Versões:** Envios subsequentes para uma mesma parametrização não sobrescrevem registros antigos. Uma nova `DocumentVersion` é inserida no histórico. O sistema garante, por meio de transação, que **apenas uma única versão** mantenha a flag de status como `ACTIVE` ao mesmo tempo, alterando as versões predecessoras automaticamente para `ARCHIVED`.
- **Evolução de Estado:** O status operacional de um `EmployeeDocument` evolui transacionalmente de `PENDING` para `COMPLETED` no momento em que a primeira versão do documento correspondente é validada.

Garantia de Atomicidade via Transações Complexas

Operações sensíveis como o vínculo de documentos obrigatórios e o recebimento de atualizações de arquivos utilizam a API de transações sequenciais do Prisma (`prisma.$transaction`). Isso garante que, em caso de erro físico ou falta de consistência ao salvar uma nova versão, toda a cadeia de operações seja revertida (Rollback), evitando estados corrompidos no banco.

7. Matriz de Endpoints da API REST

Módulo	Método	Rota	Ação Esperada / Descrição
Employees	POST	<code>/employees</code>	Registra um novo colaborador na base de dados
Employees	GET	<code>/employees</code>	Lista colaboradores filtrados com suporte a paginação
Employees	GET	<code>/employees/:id</code>	Recupera os detalhes completos de um colaborador específico
Employees	PATCH	<code>/employees/:id</code>	Atualiza parcialmente atributos de um colaborador
Employees	DELETE	<code>/employees/:id</code>	Executa a deleção lógica (Soft Delete) do colaborador
Document Types	POST	<code>/document-types</code>	Cadastra uma nova tipagem de documento obrigatório
Document Types	GET	<code>/document-types</code>	Retorna todas as tipagens cadastradas e ativas
Document Types	GET	<code>/document-types/:id</code>	Detalha um tipo de documento específico
Document Types	PATCH	<code>/document-types/:id</code>	Modifica parâmetros da tipagem documental
Document Types	DELETE	<code>/document-types/:id</code>	Desativa logicamente a tipagem especificada
Employee Docs	POST	<code>/employee-documents/:employeeId/link</code>	Vincula uma obrigatoriedade documental a um colaborador

Módulo	Método	Rota	Ação Esperada / Descrição
Employee Docs	DELETE	<code>/employee-documents/:employeeId/:documentTypeId</code>	Remove de forma lógica o vínculo de obrigatoriedade
Employee Docs	GET	<code>/employee-documents/:employeeId</code>	Exibe a grade de documentos e o status real de um funcionário
Employee Docs	GET	<code>/employee-documents/pending</code>	Lista de pendências global sob filtros dinâmicos e paginação
Documents	POST	<code>/documents/send</code>	Processa o envio de um arquivo, criando ou subindo versão

8. Guia Operacional de Execução (Ambiente Local & Docker)

Siga o passo a passo estrito para preparar o ambiente de desenvolvimento, configurar o ecossistema Docker para o banco de dados PostgreSQL e sincronizar o Prisma ORM na versão informada (`^6.19.3`).

Passo 1: Instalação das Dependências do Projeto

Efetue a instalação limpa das árvores de pacotes definidas no arquivo de manifesto da aplicação:

```
npm install
```

Passo 2: Configuração da Infraestrutura via Docker Compose

Utilize o arquivo de configuração declarativa abaixo para provisionar o container isolado do banco de dados relacional. Certifique-se de que a porta local `5432` não está ocupada por outra instância de banco nativa.

```
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    container_name: postgres
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: document_manager
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

Inicie o serviço em segundo plano (modo daemon) executando o comando abaixo na raiz do seu terminal:

```
docker-compose up -d
```

Passo 3: Configuração das Variáveis de Ambiente

Crie um arquivo nomeado estritamente como `.env` na raiz do projeto e configure a string de conexão JDBC direcionada para a instância do container Docker inicializada no passo anterior:


```
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/document_manager?
schema=public"
```

Passo 4: Sincronização do Prisma ORM e Execução de Migrations

Com o banco de dados ativo e saudável dentro do Docker, execute os comandos do CLI do Prisma (garantindo o alinhamento com a versão instalada `^6.19.3`) para compilar os schemas e aplicar as definições estruturais na base de dados:

Atenção ao Ciclo de Migrações:

Sempre utilize o comando de desenvolvimento para gerar histórico rastreável em arquivos físicos de migração dentro da pasta `prisma/migrations`.

```
# 1. Gera o cliente do Prisma fortemente tipado baseado no schema.prisma
npx prisma generate

# 2. Cria a estrutura física de tabelas e aplica as deliberações de tipos no Postgres
npx prisma migrate dev --name init
```

Passo 5: Inicialização da API em Modo Desenvolvimento

Coloque o servidor NestJS em execução com observabilidade ativa a alterações de arquivos em tempo real (Hot Reload):

```
npm run start:dev
```

Passo 6: Execução da Suíte de Testes Automatizados

Rode os testes unitários focados nas camadas de serviços de negócios da aplicação com isolamento completo através de mocks:

```
npm test
```

9. Arquitetura de Testes Unitários

A validação lógica de integridade do código ocorre no nível das camadas de serviços (`EmployeeService`, `DocumentTypeService`, etc.). O PrismaClient é injetado sob um padrão de simulação (Mock), impedindo interações diretas com o banco de dados PostgreSQL durante a suíte de testes. Isso confere velocidade extrema e idempotência aos testes de unidade controlados via Jest.

10. Cronograma de Melhorias Futuras (Roadmap)

- **Segurança:** Autenticação robusta via JWT com controle de acesso baseado em funções (RBAC).
- **Armazenamento:** Integração com Amazon S3 ou storages compatíveis com a API S3 para persistência real de binários.
- **Observabilidade:** Integração nativa com ferramentas do ecossistema APM (OpenTelemetry, Prometheus e Grafana).
- **Documentação Dinâmica:** Implementação do módulo Swagger (`@nestjs/swagger`) exposta no endpoint `/api/docs` .
- **Resiliência:** Filas de mensageria assíncronas para processamento pesado de arquivos e cache estratégico em memória via Redis.